Advanced Digital Logic

ENGR – UH 2310

Spring 2025

Instructor: Muhammad Hassan Jamil

Demarce Williams (dsw9740)
Aman Sunesh (as18181)

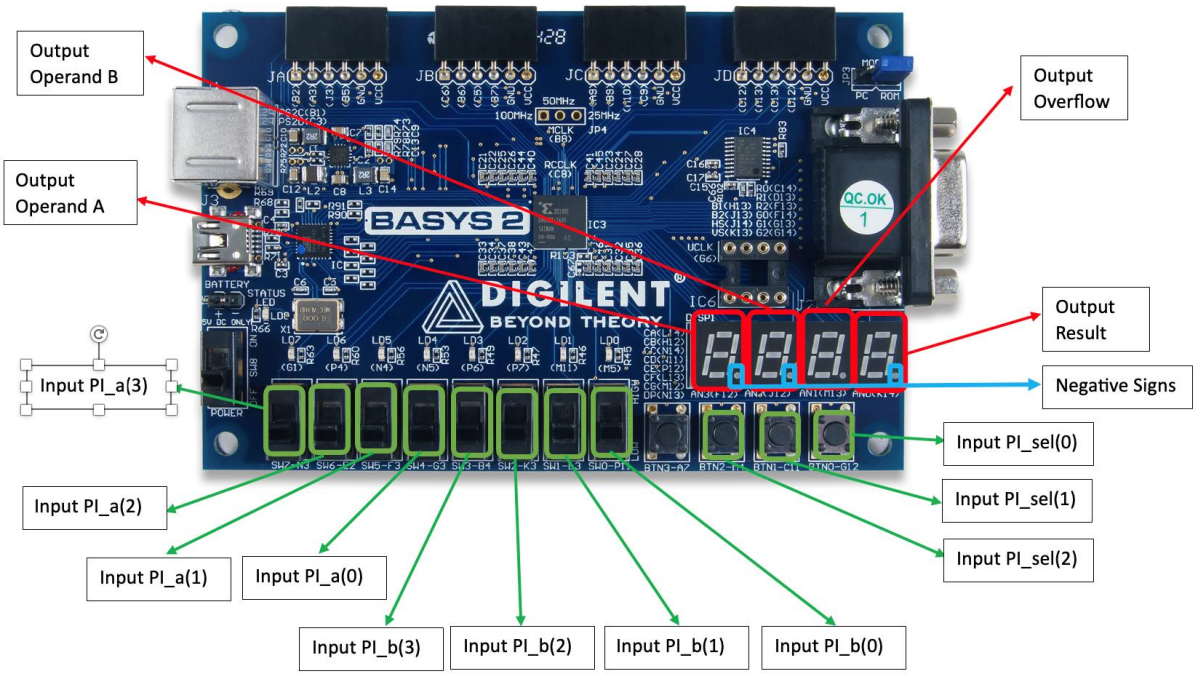
UCF WITH SCHEMATIC (CONTRAINT FLLE):

```
NET "PO_seg<0>" LOC = "L14"; # Bank = 1, Signal name = CA
NET "PO_seg<1>" LOC = "H12"; # Bank = 1, Signal name = CB
NET "PO_seg<2>" LOC = "N14"; # Bank = 1, Signal name = CC
NET "PO_seg<3>" LOC = "N11"; # Bank = 2, Signal name = CD
NET "PO_seg<4>" LOC = "P12"; # Bank = 2, Signal name = CE
NET "PO_seg<5>" LOC = "L13"; # Bank = 1, Signal name = CF
NET "PO_seg<6>" LOC = "M12"; # Bank = 1, Signal name = CG
NET "PO_seg<7>" LOC = "N13"; # Bank = 1, Signal name = DP
NET "PO_an<3>" LOC = "K14"; # Bank = 1, Signal name = AN3
NET "PO_an<2>" LOC = "M13"; # Bank = 1, Signal name = AN2
NET "PO_an<1>" LOC = "J12"; # Bank = 1, Signal name = AN1
NET "PO_an<0>" LOC = "F12"; # Bank = 1, Signal name = AN0

NET "PI_a[3]" LOC = "N3";   # Bit 3 (most significant bit) of the PI_a bus is connected to pin N3.
NET "PI_a[2]" LOC = "E2";   # Bit 2 of PI_a is assigned to pin E2.
NET "PI_a[1]" LOC = "F3";   # Bit 1 of PI_a is mapped to pin F3.
NET "PI_a[0]" LOC = "G3";   # Bit 0 (least significant bit) of PI_a is connected to pin G3.
NET "PI_b[3]" LOC = "B4";   # Bit 3 of PI_b is mapped to pin B4.
NET "PI_b[2]" LOC = "K3";   # Bit 2 of PI_b is assigned to pin K3.
NET "PI_b[1]" LOC = "L3";   # Bit 1 of PI_b is connected to pin L3.
NET "PI_b[0]" LOC = "P11";  # Bit 0 of PI_b is assigned to pin P11.
NET "PI_sel[2]" LOC = "M4"; # Bit 2 of the PI_sel bus (used to select the ALU operation) is mapped to pin M4.
NET "PI_sel[1]" LOC = "C11";# Bit 1 of PI_sel is connected to pin C11.
NET "PI_sel[0]" LOC = "G12";#  Bit 0 of PI_sel is assigned to pin G12.


NET "clk" LOC = "B8";       # The clock signal is connected to pin B8 on the FPGA. This provides the necessary
                            # clock input for the design.
```

LABELLED HD PICTURE OF INPUTS AND OUTPUTS ON FPGA BOARD FOR FSM TAIL LIGHTS CONTROLLER



Note:

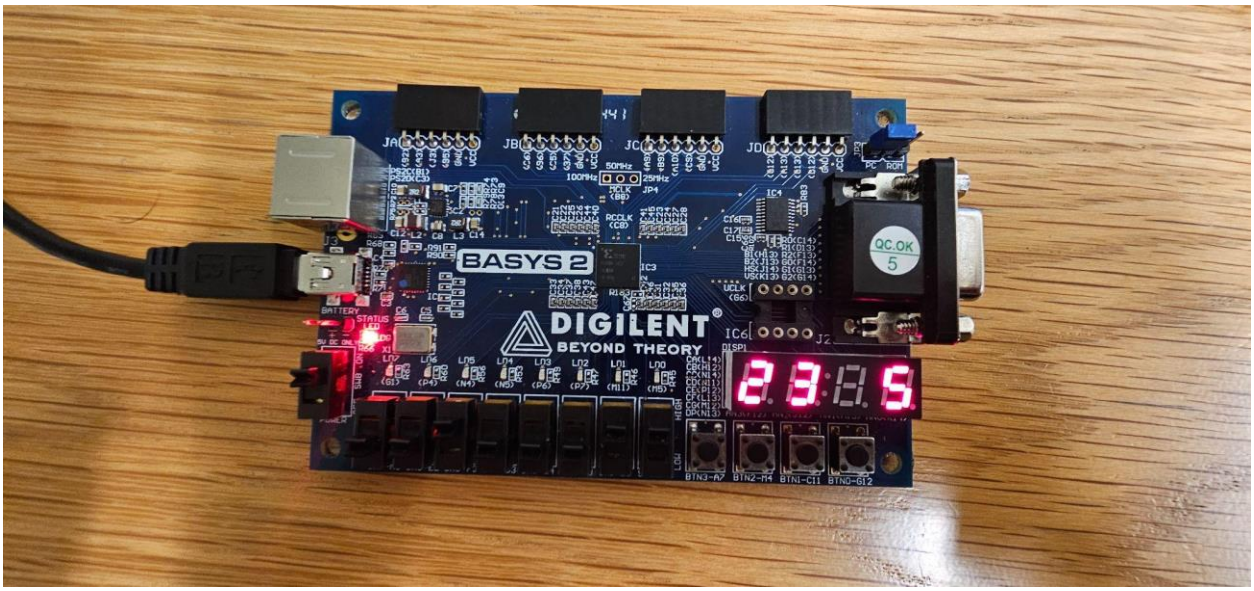
Output Operand A corresponds to the 4-bit input PI_a and Output Operand B corresponds to the 4-bit input PI_b.

For Operations: ROR (Rotate right), SLL (Shift Left Logical), SRL (Shift Right Logical), and SRA (Shift Right Arithmetic), the 4-bit input PI_a is used for the binary operand and the 4-bit input PI_b is used to store the number of shifts that must be performed on the operand.

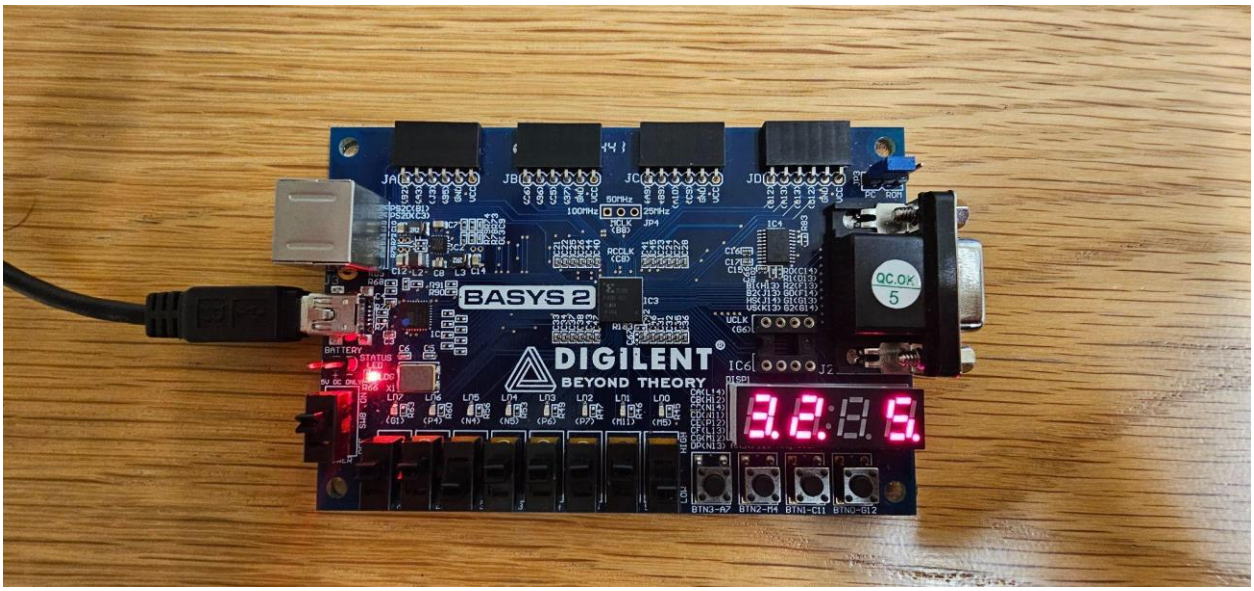
TEST CASES:

(i) ADD (on signed numbers)

a) **Input A:** 0010 (+2)
Input B: 0011 (+3)
Operation: 0010 + 0011
Output: 0101 (+5)

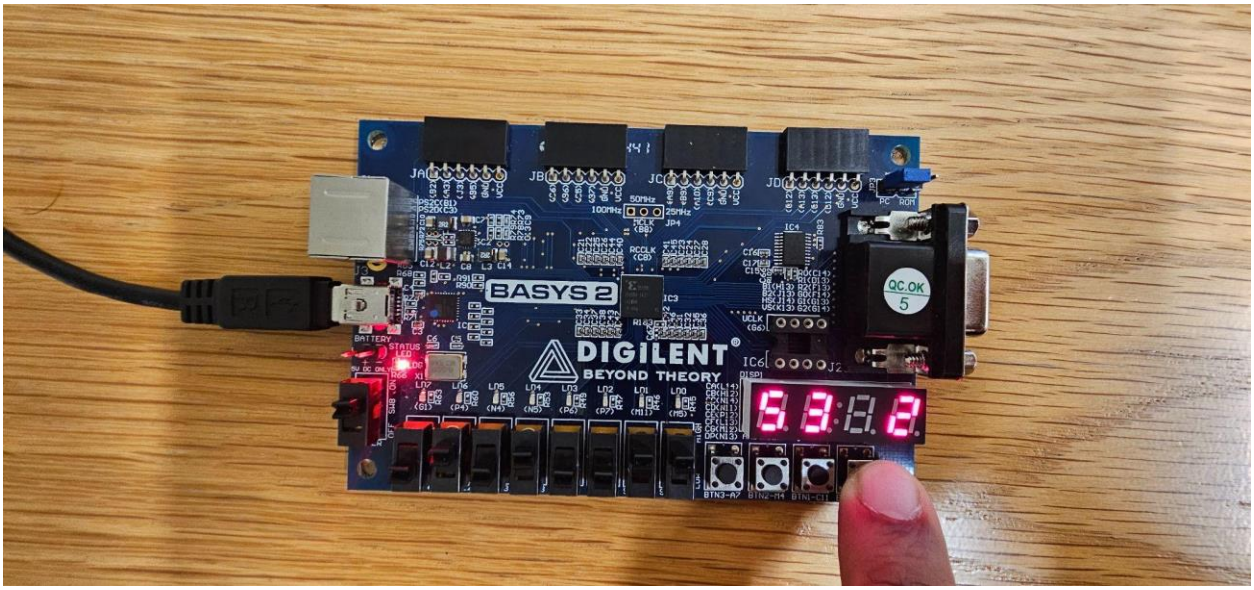


b) **Input A:** 1101 (–3; since 1101 = 13 – 16)
Input B: 1110 (–2; since 1110 = 14 – 16)
Operation: 1101 + 1110
Output: 1011 (–5; because 11 – 16 = –5)

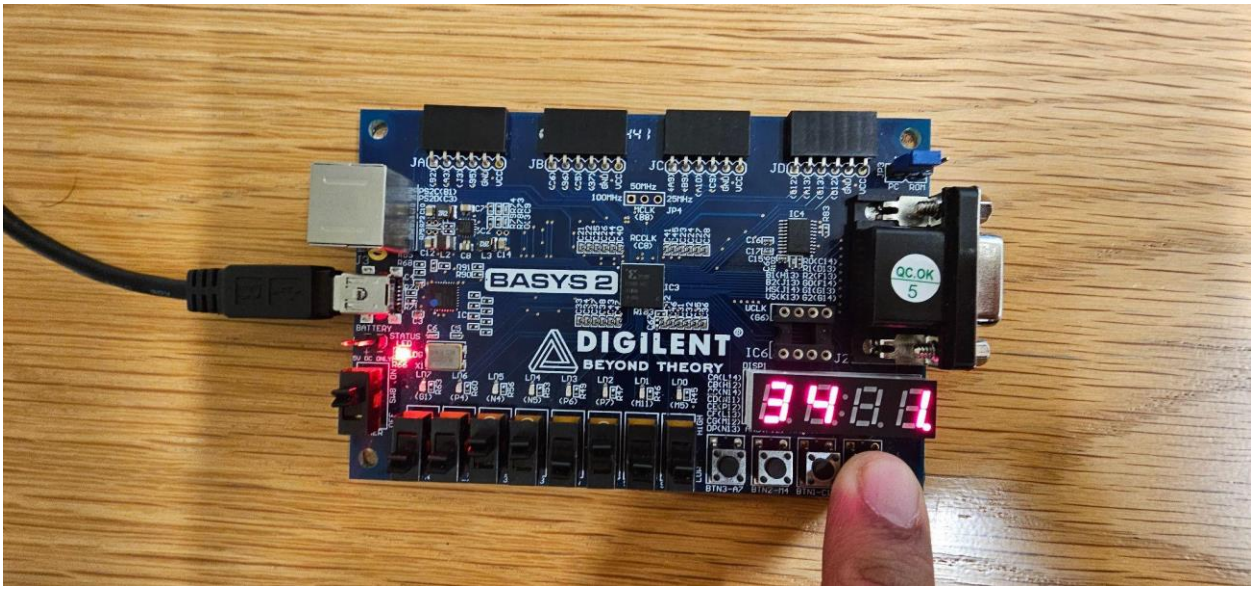


(ii) SUB (on signed numbers)

a) **Input A:** 0101 (+5)
Input B: 0011 (+3)
Operation: 0101 – 0011
Output: 0010 (+2)

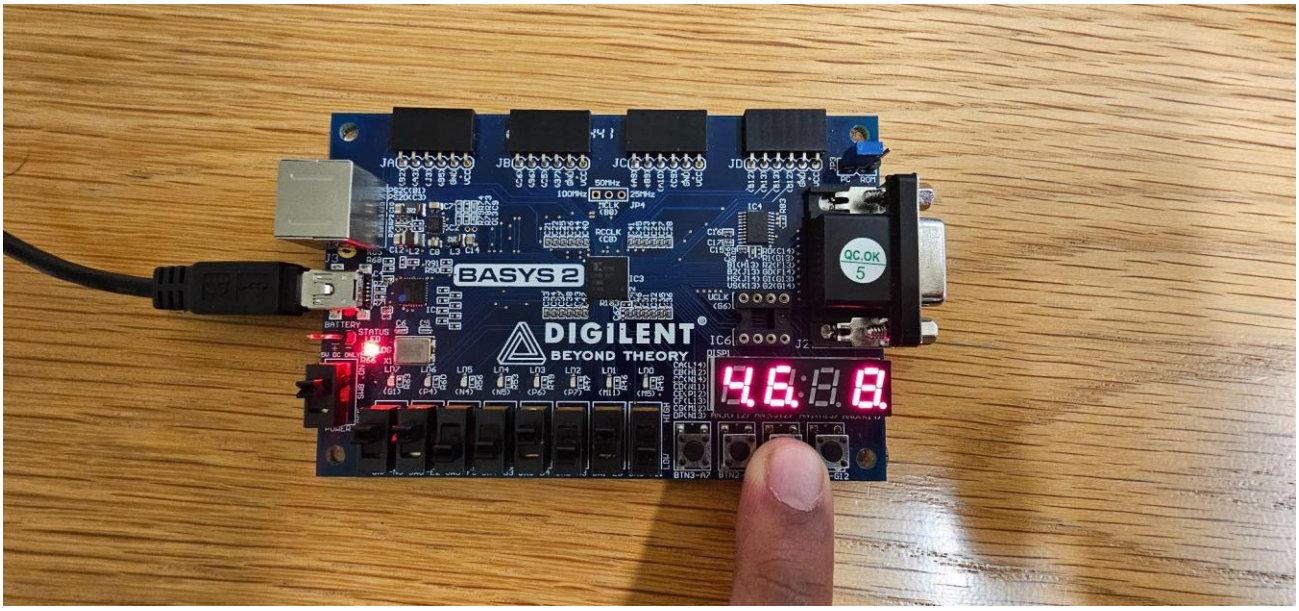


b) **Input A:** 0011 (+3)
Input B: 0100 (+4)
Operation: 0011 – 0100
Output: 1111 (–1; because 15 – 16 = –1)

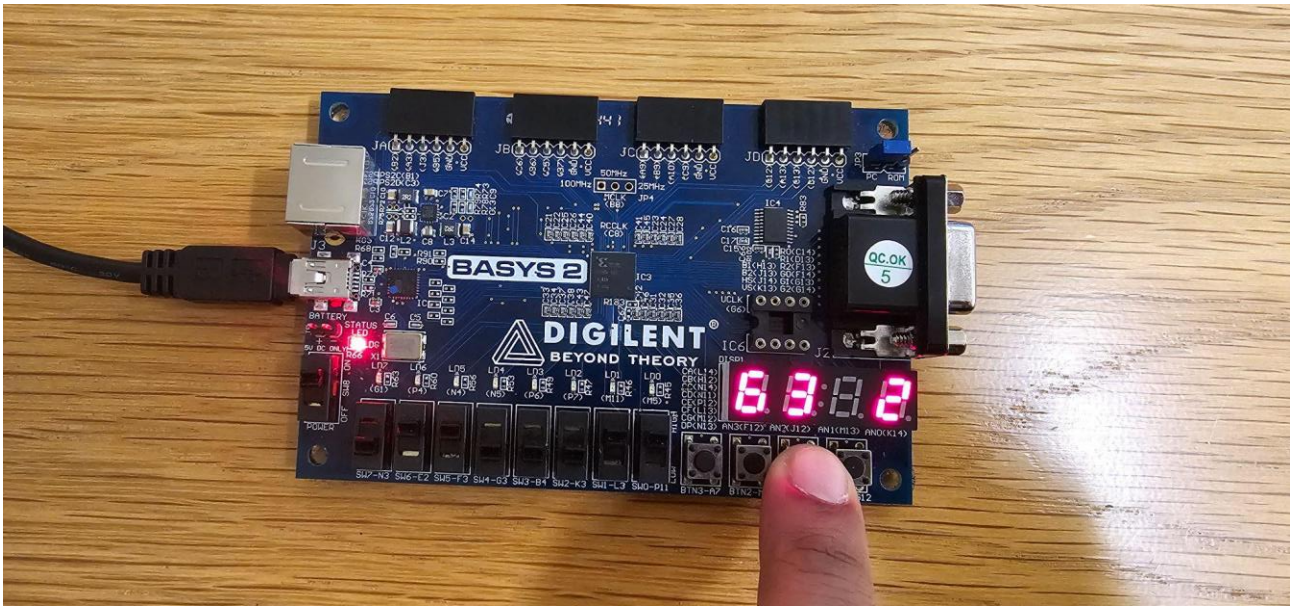


(iii) AND (bitwise)

a) **Input A:** 1100 (-4)
Input B: 1010 (-6)
Operation: 1100 AND 1010
Output: 1000 (-8)

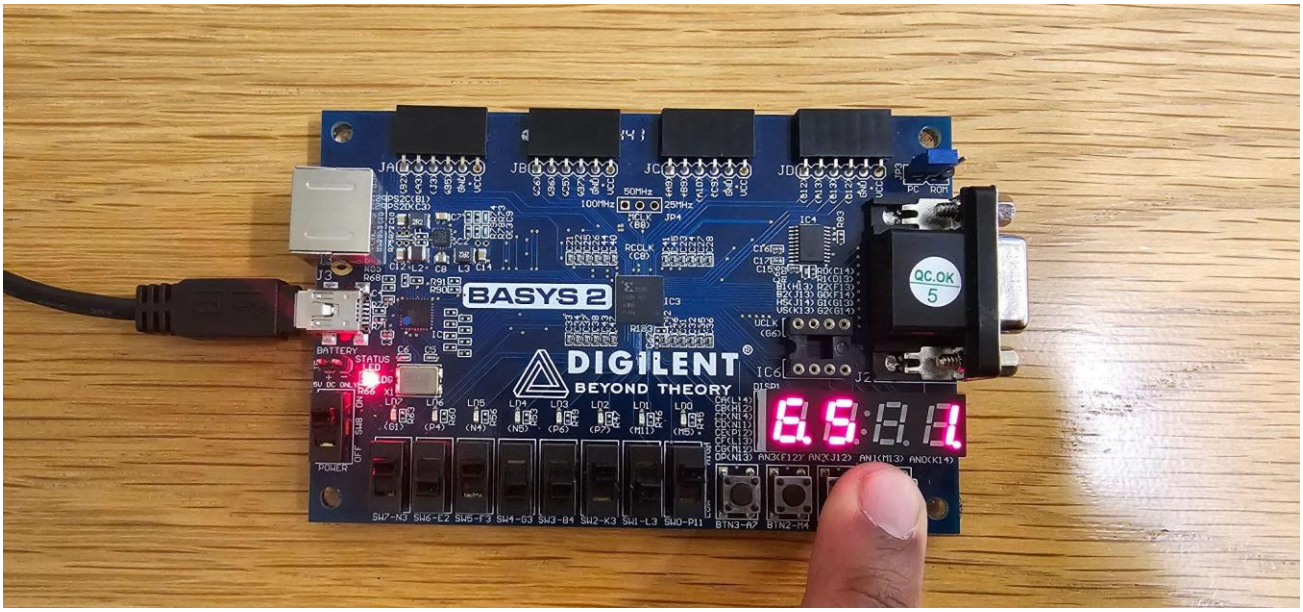


b) **Input A:** 0110 (+6)
Input B: 0011 (+3)
Operation: 0110 AND 0011
Output: 0010 (+2)

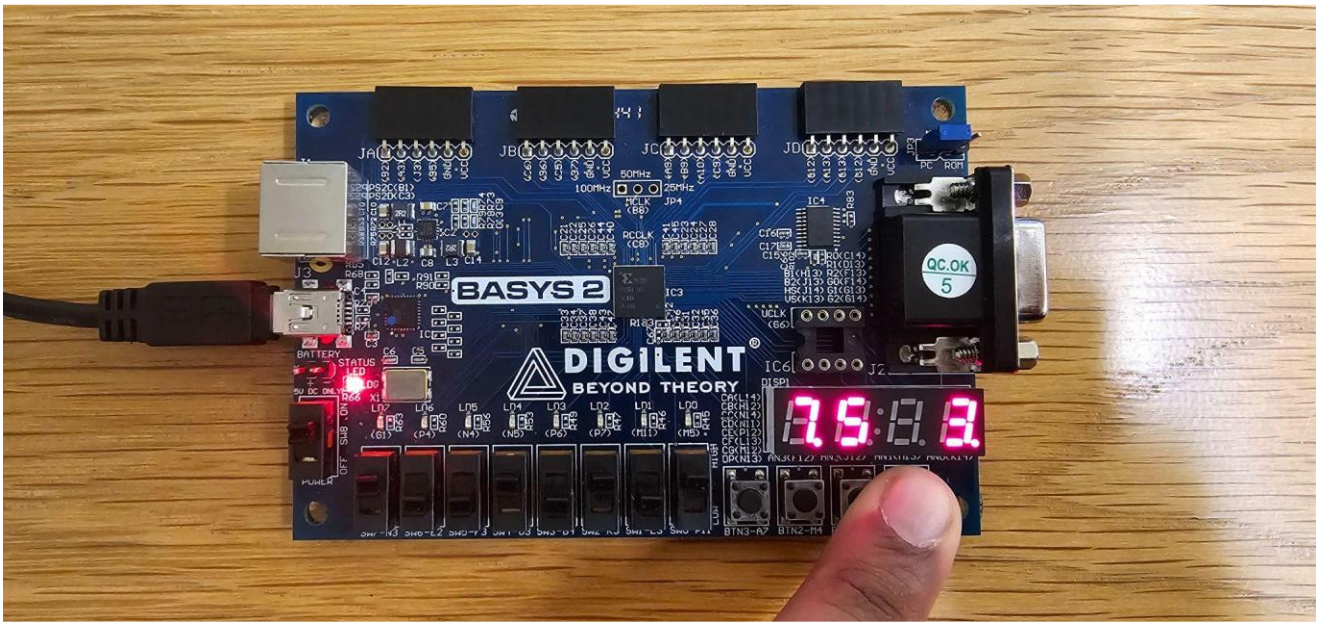


(iv) OR (bitwise)

a) **Input A:** 1010 (-6)
Input B: 0101 (+5)
Operation: 1010 OR 0101
Output: 1111 (-1)



b) **Input A:** 1001 (-7)
Input B: 0101 (+5)
Operation: 1001 OR 0101
Output: 1101 (-3)



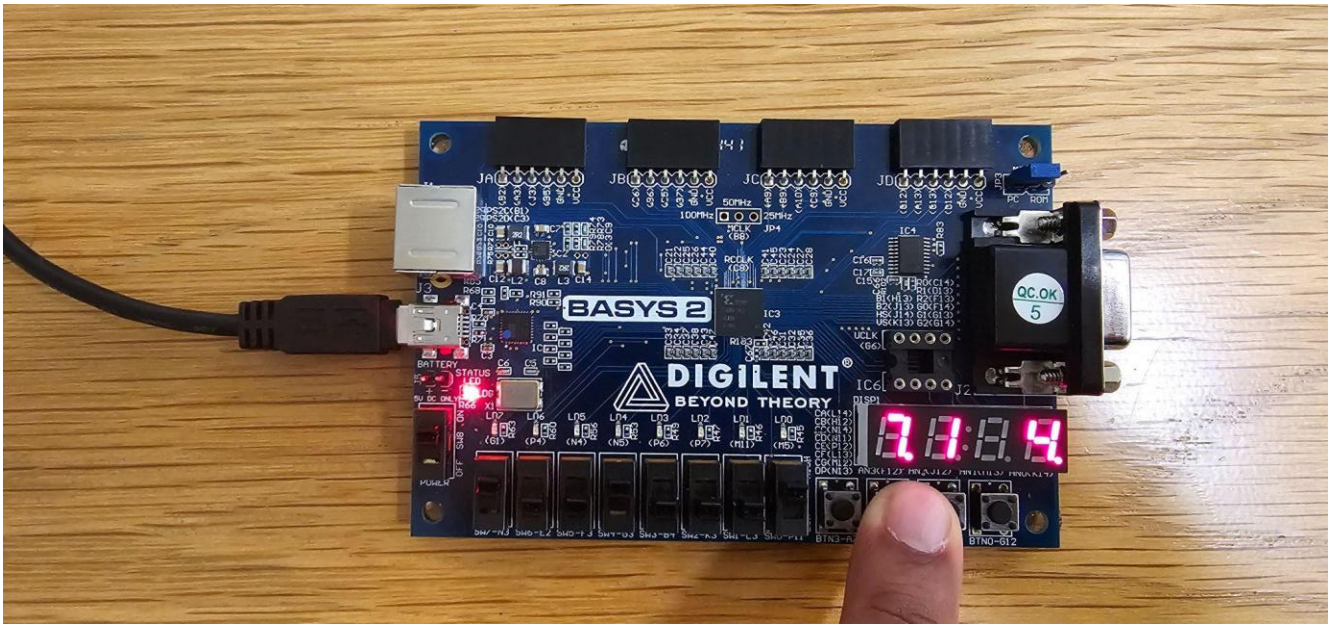
(v) ROR (Rotate right)

a) **Input A:** "1001" (-7)
Input B: "0001" (rotate by 1) (+1)

Calculation:

- Original: $a_3a_2a_1a_0$ = 1 0 0 1
- After rotation: becomes $a_0a_3a_2a_1$ = **1 1 0 0**

Expected Result: "1100" (→ 8-bit: "00001100") (-4)

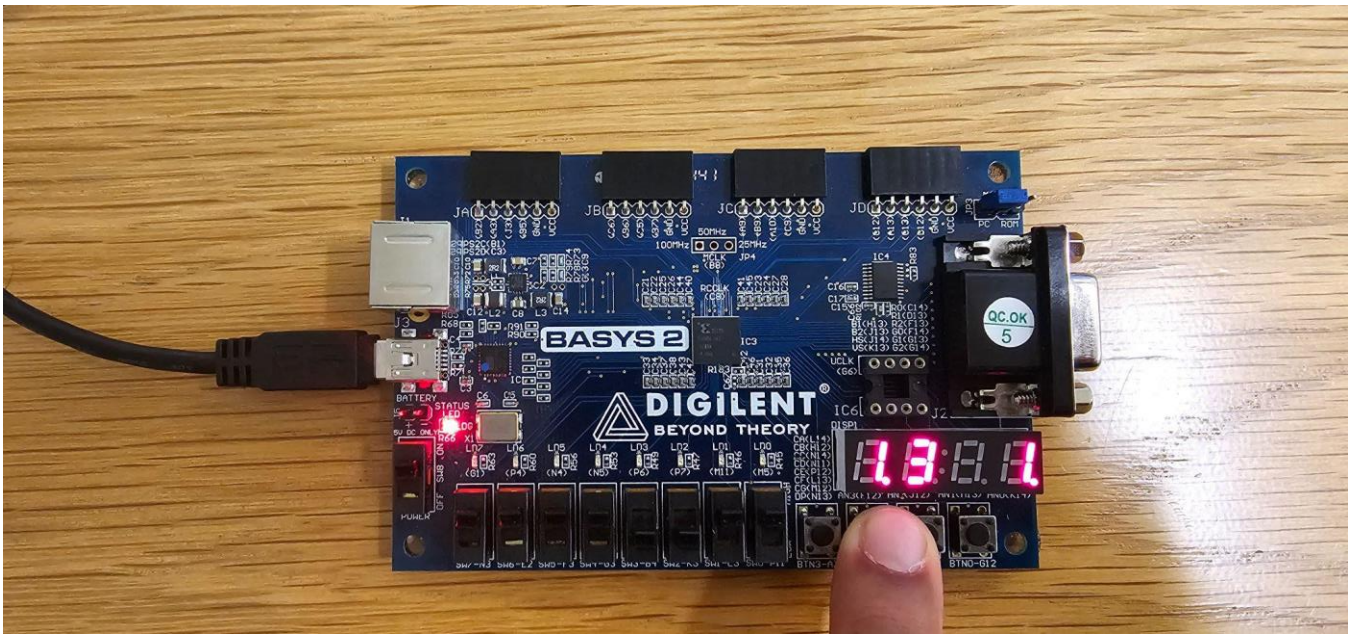


b) **Input A:** "1111" (-1)
Input B: "0011" (rotate by 3) (+3)

Calculation:

- Rotating all ones always yields all ones.

Expected Result: "1111" (→ 8-bit: "00001111") (-1)



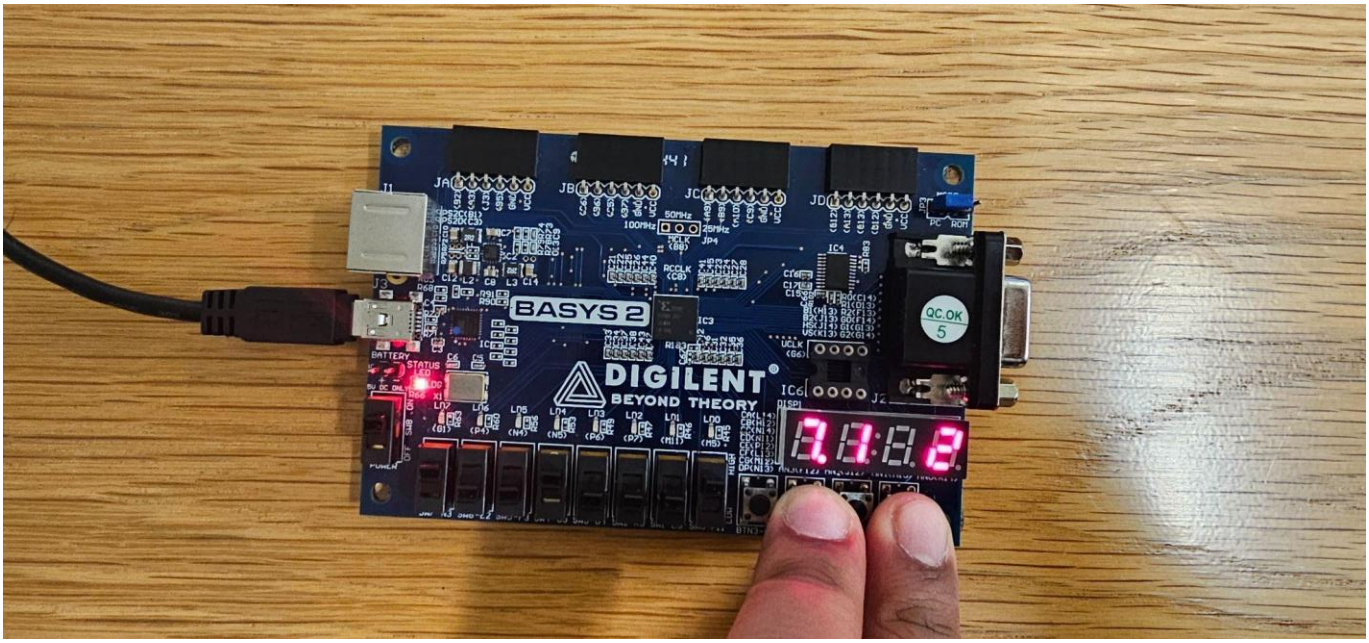
(vi) SLL (Shift Left Logical)

a) **Input A:** "1001" (-7)
Input B: "0001" (shift left by 1) (+1)

Calculation:

- "1001" shifted left by 1: drop the MSB and append a 0 → **"0010"**
- (9 × 2 = 18; modulo 16 gives 2, which is "0010")

Expected Result: "0010" (→ 8-bit: "00000010") (+2)

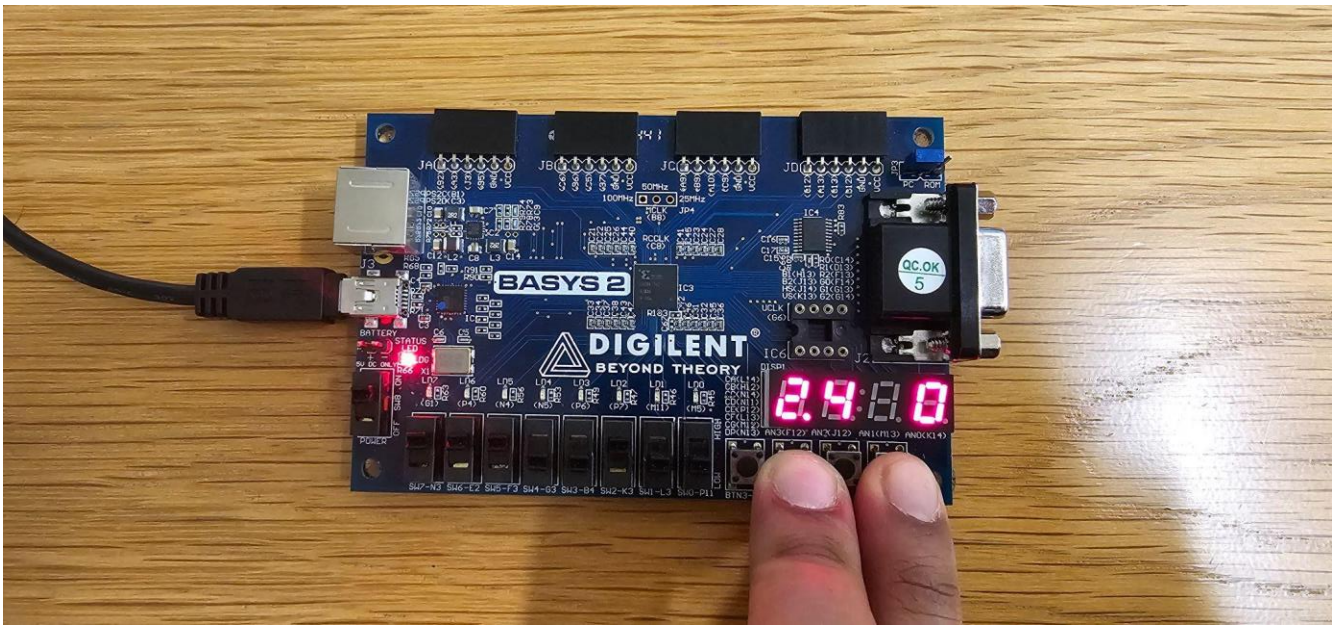


b) **Input A:** "1110" (-2)
Input B: "0100" (shift left by 4) (+4)

Calculation:

- Shifting by the full width (4 bits) clears all bits → "0000"

Expected Result: "0000" (→ 8-bit: "00000000") (0)

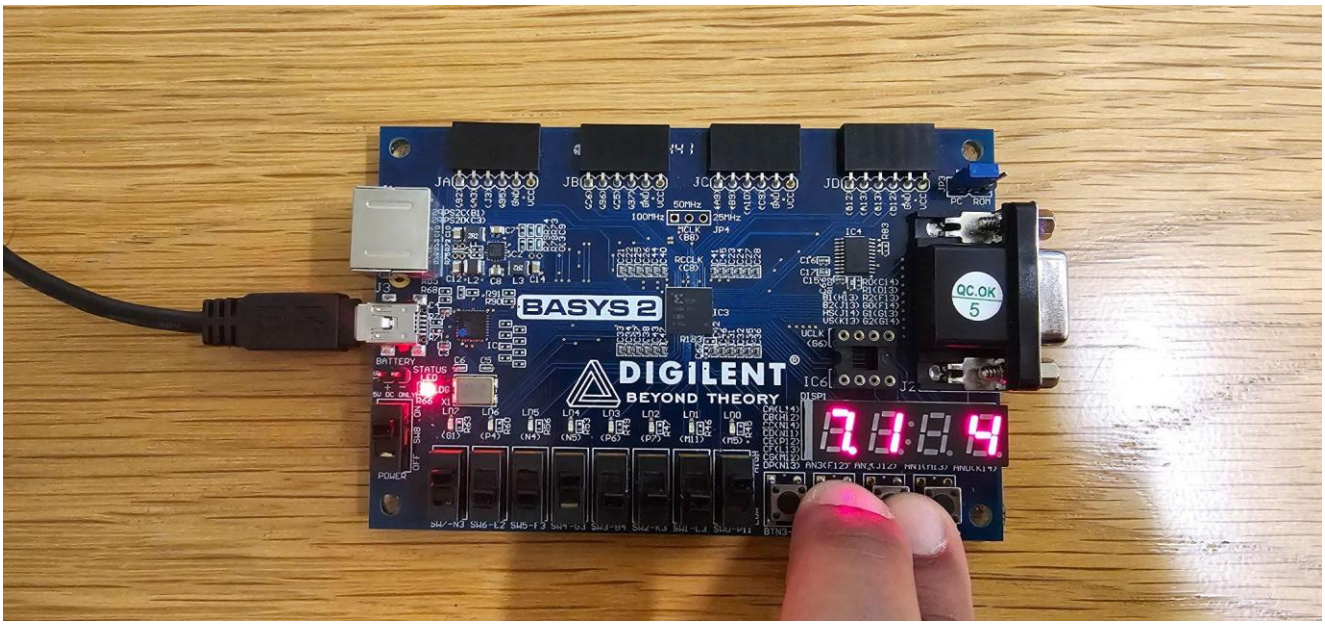


(vii) SRL (Shift Right Logical)

a) **Input A:** "1001" (-7)
Input B: "0001" (shift right by 1) (+1)
Calculation:

- "1001" shifted right logically by 1: insert 0 on the left → **"0100"**
- (9 ÷ 2 = 4, which is "0100")

Expected Result: "0100" (→ 8-bit: "00000100") (+4)

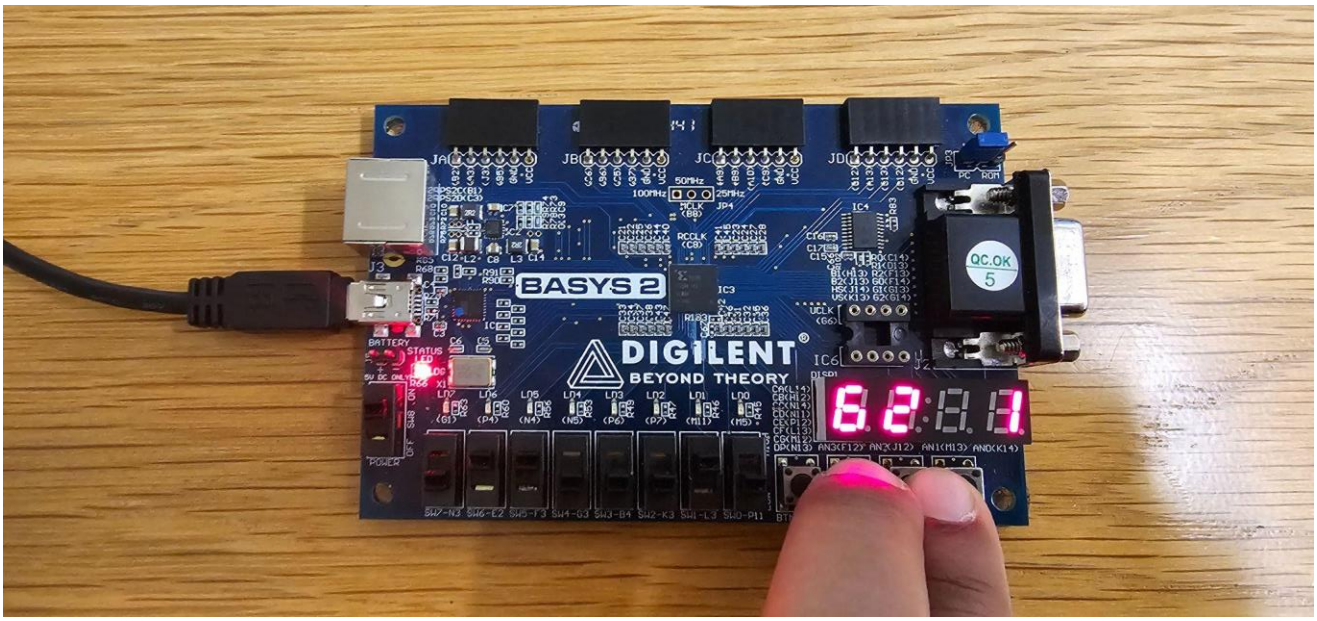


b) **Input A:** "0110" (+6)
Input B: "0010" (shift right by 2) (+2)

Calculation:

- "0110" (6) shifted right by 2: 6 ÷ 4 = 1 (ignoring fractions) → **"0001"**

Expected Result: "0001" (→ 8-bit: "00000001") (+1)



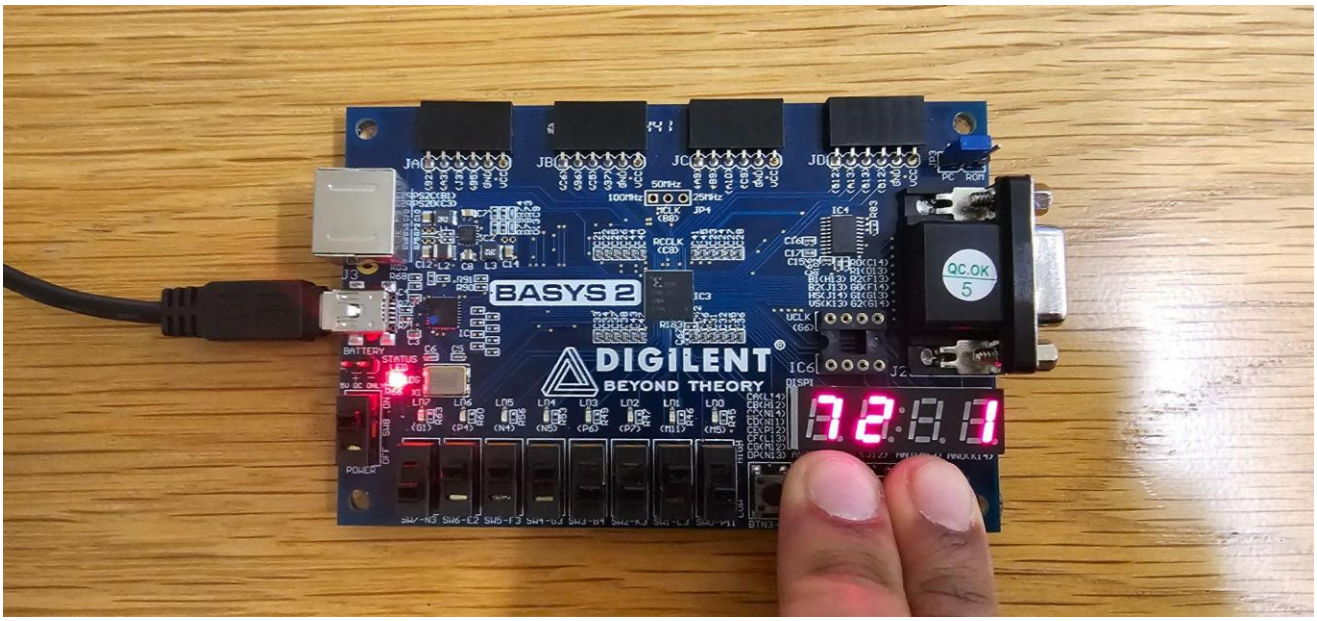
(viii) SRA (Shift Right Arithmetic)

a) **Input A:** "0111" (+7)
Input B: "0010" (shift right by 2) (+2)

Calculation:

- For positive numbers, arithmetic shift equals logical shift: 7 ÷ 4 = 1 → **"0001"**

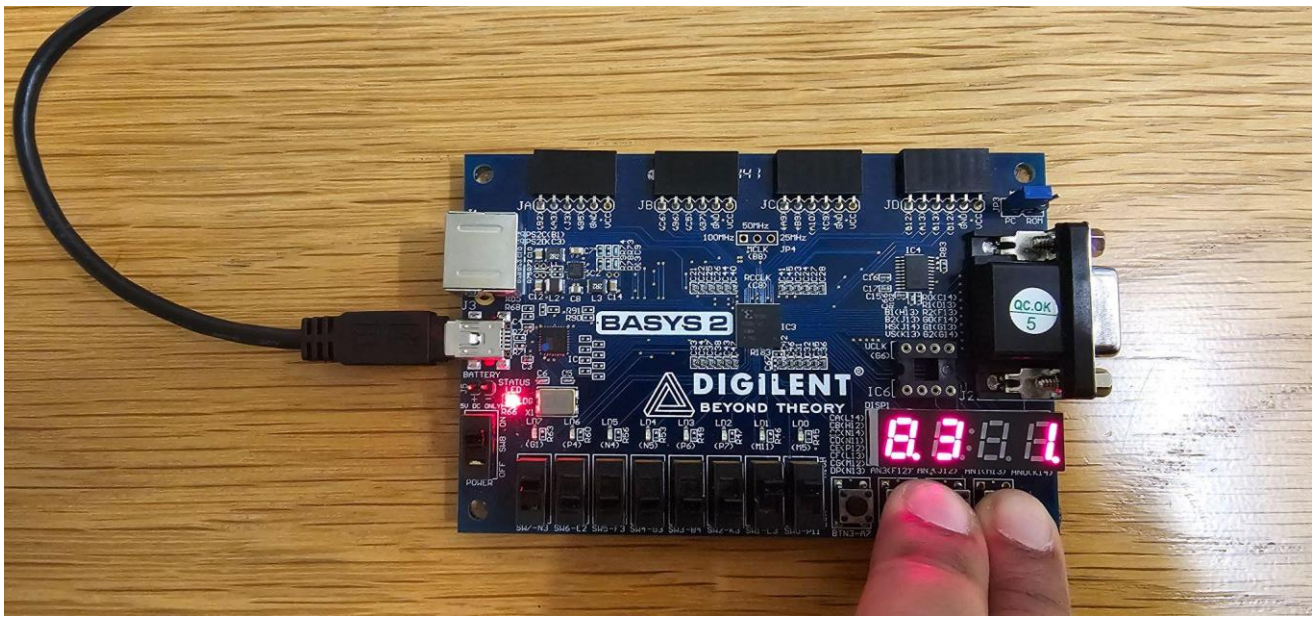
Expected Result: "0001" (→ 8-bit: "00000001") (+1)



b) **Input A:** "1000" (-8)
Input B: "0011" (shift right by 3) (+3)
Calculation:

- –8 divided by 8 equals –1. In 4-bit two's complement, –1 is **"1111"**.

Expected Result: "1111" (→ 8-bit: "00001111") (-1)

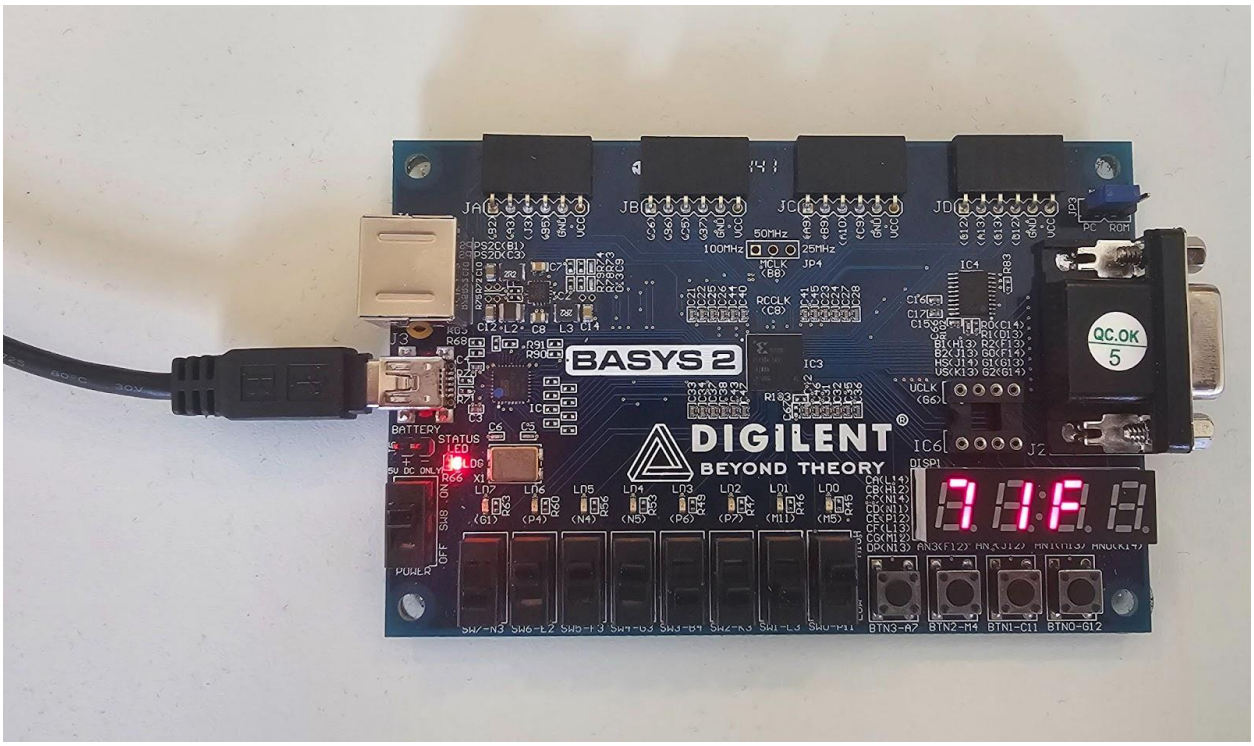


BONUS:

(i) ADD (on signed numbers)

Input A: 0111 (+7)
Input B: 0001 (+1)
Operation: 0111 + 0001
Expected Output: 7 + 1 = 8
Actual Output = -8

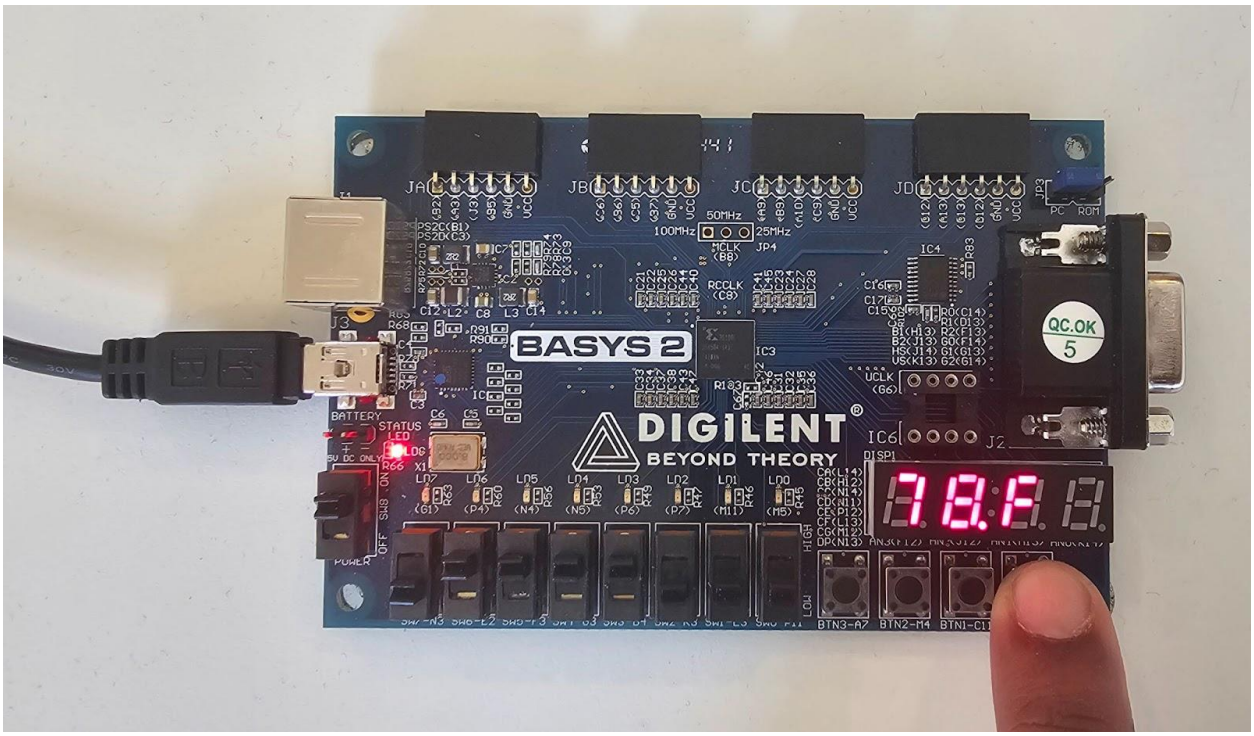
Note: Since the maximum representable value in 4-bit two's complement is 7, the sum (+8) exceeds this limit. Therefore, the ALU's overflow flag (overflow) will be set to '1'.



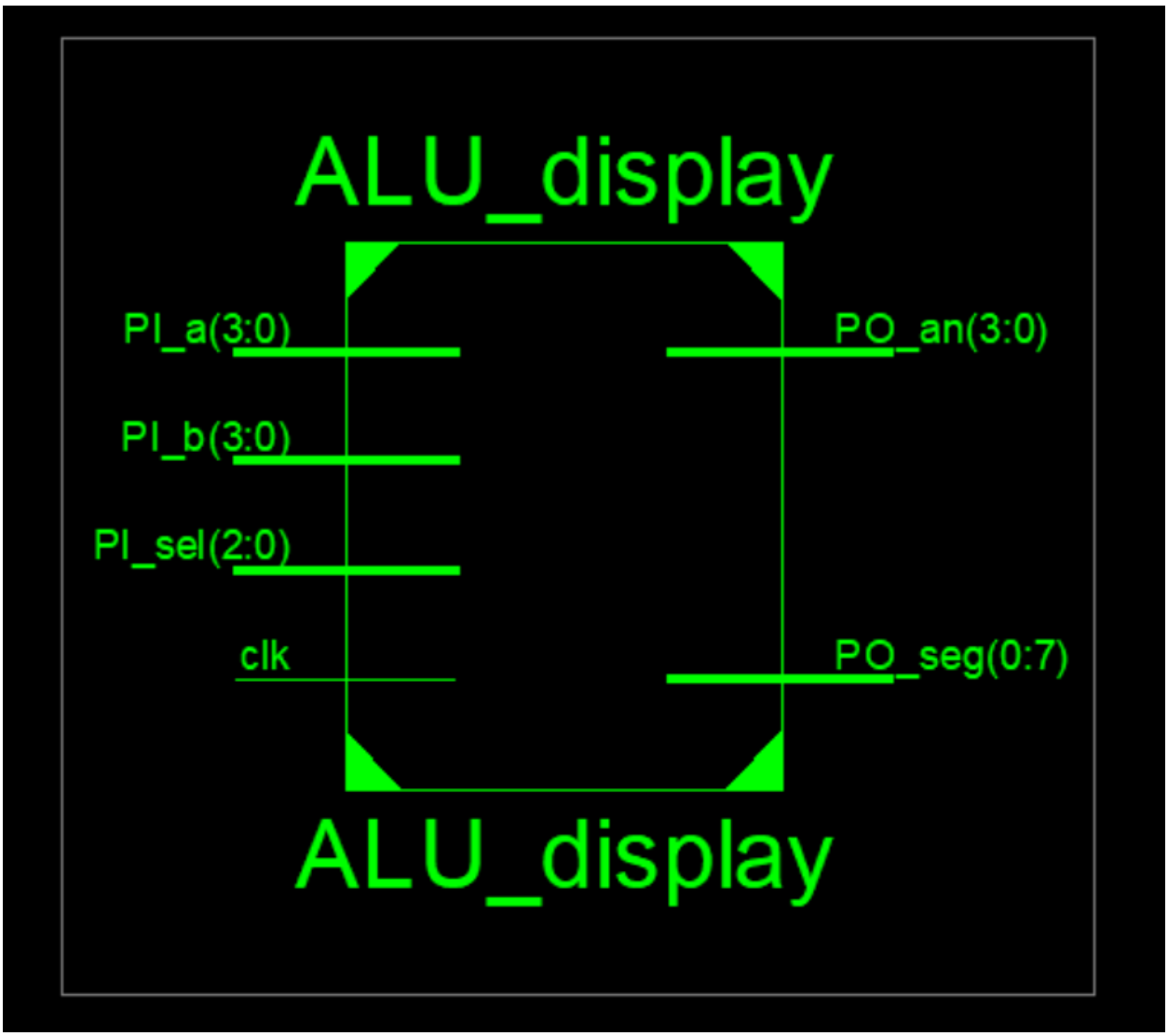
(ii) SUB (on signed numbers)

Input A: 0111 (+7)
Input B: 1000 (-8)
Operation: 0111 - 1000
Expected Output: 7 - (-8) = 15
Actual Output = -1

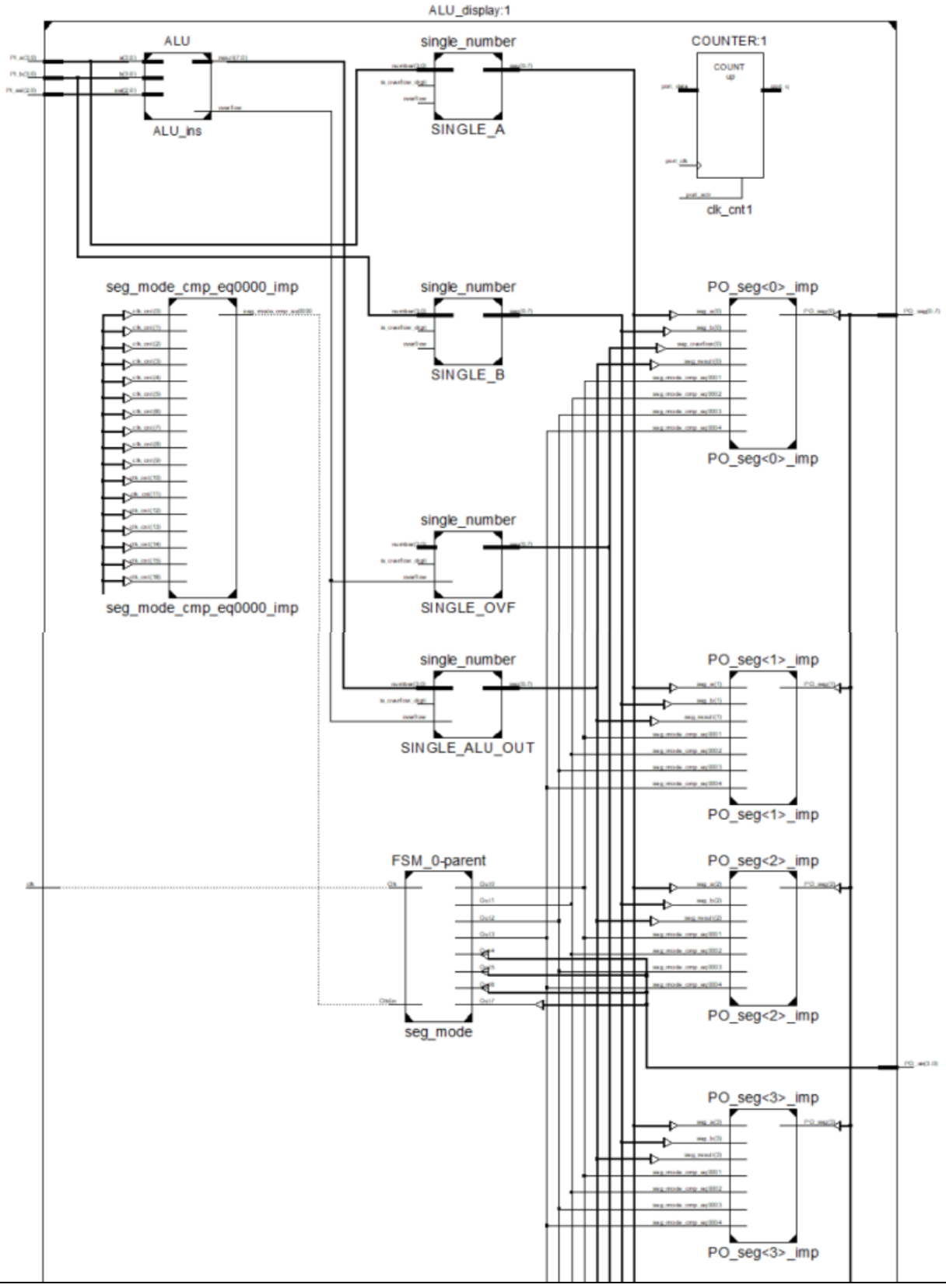
Note: The result 15 is outside the representable range (–8 to 7) for a 4-bit two's complement number. Thus, the ALU detects an overflow and sets the overflow flag to '1'.

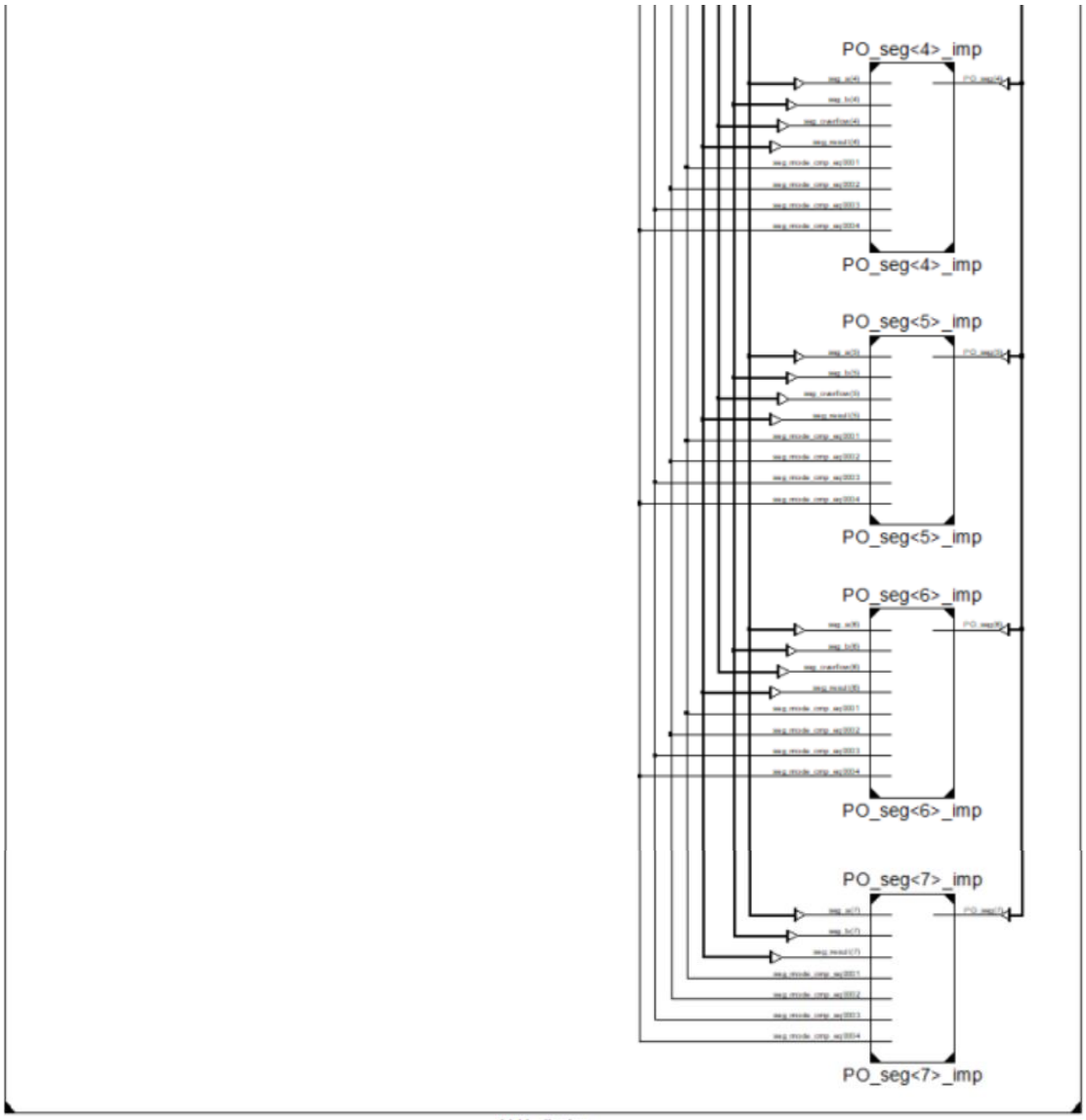


RTL Schematic Block Diagram



RTL Schematic Top Level (Implementation)




PO_seg<4>_imp
PO_seg<4>_imp
PO_seg<5>_imp
PO_seg<5>_imp
PO_seg<6>_imp
PO_seg<6>_imp
PO_seg<7>_imp
PO_seg<7>_imp
ALU_display